

Universidad Nacional Autónoma de México
Facultad de Ingeniería
Fundamentos de Sistemas Embebidos
Proyecto Final: Consola de videojuegos Retro Embebida

Alumno: Monter González Luis Enrique

Grupo: 02

Fecha: 3 de junio de 2026

Video demostrativo: https://youtu.be/Xch_X6m8AQw

Repositorio: <https://github.com/daniel13nic/proyecto-retro-embebidos>

1. Introducción

El diseño de sistemas embebidos requiere una integración profunda entre el hardware disponible y el sistema operativo que lo administra. A diferencia de las computadoras de propósito general, los sistemas embebidos están diseñados para cumplir una función específica con recursos computacionales limitados, lo que exige la eliminación de capas de software innecesarias [1].

La implementación de una consola de videojuegos retro sobre una arquitectura ARM representa un desafío técnico enfocado en la optimización. El presente proyecto detalla la construcción de un sistema autónomo capaz de emular arquitecturas clásicas interactuando directamente con el hardware de entrada y salida, prescindiendo totalmente de la intervención de un entorno de escritorio convencional. A lo largo de este reporte, se abordará la preparación física del dispositivo, el desarrollo de una interfaz gráfica nativa interactiva, la automatización del montaje de almacenamiento masivo y la orquestación del flujo de ejecución desde el arranque del sistema.

2. Antecedentes Teóricos

Para comprender la arquitectura de la solución propuesta, es necesario definir los componentes lógicos que actúan a nivel de sistema operativo:

- **Kernel Mode Setting (KMS) y Direct Rendering Manager (DRM):** En los sistemas Linux, el servidor de visualización actúa como intermediario entre las aplicaciones y el búfer gráfico. El protocolo KMS/DRM permite a los programas comunicarse directamente con el controlador de la GPU en el kernel [3]. Su uso es fundamental en sistemas embebidos para liberar memoria RAM y minimizar latencias de renderizado.
- **Systemd:** Es el administrador de sistemas y servicios predeterminado en distribuciones modernas de Linux. Permite paralelizar el inicio de servicios, gestionar dependencias en el arranque y asignar terminales virtuales (`ttty`) para la ejecución exclusiva de procesos [1].

- **Udev:** Es el gestor de dispositivos del kernel. Opera en el espacio de usuario y escucha de forma asíncrona las interrupciones de hardware generadas en los buses físicos, permitiendo ejecutar acciones automatizadas basadas en reglas [2].
- **Comunicación Interprocesos (IPC):** Dado que los sistemas embebidos poseen recursos volátiles limitados, la exclusión mutua entre procesos pesados es obligatoria. El uso de archivos temporales en el sistema de archivos permite que dos programas intercambien información de estado sin ejecutarse simultáneamente en memoria.

3. Objetivo

Implementar una consola de videojuegos retro como sistema embebido, capaz de arrancar directamente a una interfaz de usuario propia, operar sin teclado ni mouse, emular juegos de NES, SNES y Game Boy Advance, e importar ROMs desde una memoria USB al almacenamiento local de la tarjeta controladora de forma interactiva.

4. Materiales y Componentes Electrónicos

Para la replicación de este prototipo, es necesario contar con la siguiente instrumentación de hardware, incluyendo sus posibles equivalentes:

- **Tarjeta Controladora:** Raspberry Pi 4 Model B (Equivalentes: Orange Pi 5, Raspberry Pi 5 o cualquier SBC basada en arquitectura ARM de 64 bits con soporte de video nativo).
- **Alimentación:** Adaptador de corriente USB-C con salida regulada de 5V y 3A (Equivalente: Fuente de alimentación de banco de laboratorio configurada a 5V/3A).
- **Almacenamiento Primario:** Tarjeta MicroSD, mínimo 16 GB, Clase 10 o U3 (Equivalente: Módulo eMMC si la placa base lo soporta).
- **Almacenamiento Secundario:** Memoria flash USB formateada en sistemas de archivos FAT32 o exFAT.
- **Interfaz de Control:** Gamepad USB DualShock 4 (Equivalente: Controlador genérico D-Input o X-Input).
- **Periféricos de salida:** Cable microHDMI a HDMI, monitor externo y altavoces activos.

5. Descripción y Funcionamiento del Hardware

5.1. Funcionamiento de la Tarjeta Controladora

El procesamiento central se realiza en la computadora de placa reducida Raspberry Pi 4 Model B, equipada con un SoC (System on a Chip) Broadcom BCM2711. Este circuito integrado contiene un microprocesador ARM Cortex-A72 de cuatro núcleos a 1.5 GHz y un procesador gráfico VideoCore VI [3].

Para las tareas de emulación, donde se requieren temporizadores precisos para la sincronización de audio y fotogramas a 60 Hz, esta arquitectura de 64 bits proporciona la estabilidad en tiempo real requerida.

5.2. Componentes Electrónicos Relevantes

Adicional al procesador central, el sistema interactúa con integrados periféricos críticos para su operación:

- **Controlador USB (VL805):** La placa incorpora un controlador de host PCIe a USB 3.0 dedicado. Esta separación de los canales de datos permite que el sistema embebido realice transferencias masivas desde la memoria USB externa sin congestionar el bus donde se reciben las señales del Gamepad.
- **Memoria NAND Flash y Controlador:** Las tarjetas MicroSD utilizadas incorporan un microcontrolador interno que gestiona el desgaste de las celdas de memoria NAND (*wear leveling*). Como el sistema operativo Linux embebido realiza operaciones constantes de escritura, la calidad de este controlador dicta la vida útil del sistema físico.
- **Microcontrolador de Interfaz Humana (Gamepad):** El control contiene un integrado que registra los cierres de circuito de los botones y envía datos seriales mediante el protocolo USB HID (Human Interface Device) hacia la tarjeta controladora.

6. Cuidados, Límites de Operación y Advertencias de Seguridad

6.1. Cuidado de la Salud y Riesgos Operativos

La operación de dispositivos electrónicos a nivel de placa de desarrollo presenta riesgos físicos que deben evitarse:

- **Riesgo de Quemaduras:** Bajo procesamiento continuo (emulación), el encapsulado metálico del SoC BCM2711 puede alcanzar temperaturas superiores a los 80°C. No se debe tocar la superficie del microprocesador durante o inmediatamente después de su operación sin disipadores térmicos.
- **Riesgo Eléctrico (Electrocución/Cortocircuito):** Se debe asegurar que la fuente de alimentación esté aislada y conectada a un receptáculo aterrizado. Manipular la placa con las manos húmedas o sobre superficies metálicas conductoras compromete la seguridad del operador y del equipo.

6.2. Protección de Componentes Electrónicos Delicados

La manipulación directa de los circuitos exige consideraciones preventivas para evitar daños irreversibles a nivel de silicio:

- **Uso de Pulsera Anti-estática:** Los pines GPIO y las pistas del circuito impreso carecen de blindaje. La manipulación requiere el uso de una pulsera anti-estática conectada a tierra física para evitar que las descargas electrostáticas (ESD) dañen los transistores CMOS [1].

- **Acumulación de Grasa en Terminales:** Las terminales doradas de la tarjeta MicroSD y de los puertos HDMI sufren degradación de conductividad y oxidación si entran en contacto directo con los dedos, debido a la transferencia de grasa de la piel.
- **Límites de Temperatura y Alimentación:** Si el adaptador de corriente no provee 3A continuos, ocurren caídas de tensión. El procesador responde disminuyendo su frecuencia (*thermal/power throttling*), lo que vuelve inoperable la emulación fluida. Adicionalmente, el corte abrupto de energía daña la estructura de la memoria flash.

7. Ensamblaje y Conexión Física

Para asegurar que el gestor de arranque del sistema operativo reconozca los buses de datos antes de iniciar el entorno lógico, la interconexión del hardware debe realizarse mediante una secuencia estructurada:

1. Insertar la tarjeta MicroSD en la ranura MicroSD de la placa base con guantes de nitrilo o tras limpiar los contactos.
2. Conectar la terminal microHDMI de la pantalla externa en el puerto HDMI 0. Este es el bus de video primario controlado directamente por la GPU.
3. Insertar el cable del Gamepad en el bus USB 2.0 (puerto negro), reservando los puertos de alta velocidad (azules) para la posterior inserción de memorias flash.
4. Una vez asegurados los periféricos de entrada y salida, suministrar la energía conectando el puerto USB-C.



Figura 1: Disposición del hardware ensamblado mostrando la conexión simultánea de las interfaces físicas (MicroSD, bus microHDMI, USB de control y puerto de energización).

8. Configuración de la Tarjeta Controladora e Instalación

Se utiliza la imagen *Raspberry Pi OS Lite*, una variante que carece de servidor gráfico (X11) y entorno de escritorio. Esta selección se debe a la necesidad de reservar la memoria RAM exclusivamente para el emulador [3]. La prueba de los programas se realizó directamente sobre la tarjeta controladora con los periféricos conectados.

Mediante el script de configuración proporcionado (`install.sh`), se automatiza la preparación del sistema operativo, actualizando paquetes y configurando dependencias:

Listing 1: Instalación de paquetes y librerías en el entorno de desarrollo

```
1 #!/bin/bash
2 sudo apt update
3 sudo apt install -y python3 python3-pygame mednafen joystick evtest mpv rsync
```

Se instalaron `evtest` y `joystick` para administrar los módulos del kernel encargados de recibir los cambios de voltaje de los botones del mando, registrándolos en `/dev/input/`. Se requiere `rsync` para gestionar copias de archivos y `mednafen` como el software central de interpretación de ROMs [4].

9. Desarrollo de los Componentes de Software (Módulos)

El sistema operativo embebido interactúa con tres módulos de software independientes para cumplir sus funciones operativas sin generar procesos simultáneos pesados.

9.1. Módulo de Interfaz Gráfica Nativa (`menu.py`)

Este módulo, escrito en Python con la librería Pygame, funciona como interfaz visual. Para lograr su ejecución sin un escritorio de Linux, configura los parámetros del sistema para que las instrucciones de dibujo pasen directamente al hardware de video.

Listing 2: Configuración del entorno gráfico y direccionamiento de video

```
1 import os
2 import pygame
3 import sys
4
5 # Asignacion del buffer directo KMS/DRM al controlador de video
6 os.environ["SDL_VIDEODRIVER"] = "kmsdrm"
```

Al detectar una pulsación en el gamepad, el script ejecuta la lógica de exclusión mutua: guarda el nombre del juego en un archivo de texto y finaliza su propio proceso, garantizando la liberación de la RAM antes de que el procesador inicie la carga del emulador.

Listing 3: Mapeo de eventos lógicos y creación de la transacción de estado

```
1 for evento in pygame.event.get():
2     if evento.type == pygame.JOYBUTTONDOWN:
```

```

3     if evento.button == 0 and juegos:
4         ruta_juego = os.path.join(ROM_DIR, juegos[indice_seleccionado])
5
6         # Escritura de estado persistente
7         with open("/tmp/next_game.txt", "w") as f:
8             f.write(ruta_juego)
9
10        # Liberacion de la memoria volatil
11        pygame.quit()
12        sys.exit(0)

```

9.2. Módulo de Detección Dinámica y Sincronización (`sync_roms.sh`)

Se encarga de la importación lógica de las ROMs. Su comportamiento protege la integridad de la MicroSD; a través de la instrucción `rsync --ignore-existing`, evita reescribir archivos existentes en las celdas NAND. Además, este módulo congela el procesador enviando señales de detención (`kill -STOP`) a la interfaz gráfica y al emulador, asegurando el ancho de banda del bus interno durante el copiado.

9.3. Módulo Orquestador y Emulación (`start.sh`)

La orquestación de la ejecución es el bloque que otorga el comportamiento de consola al hardware. Este módulo se estructuró mediante la combinación de un script en Bash, un archivo de texto y el motor de emulación.

9.3.1. Selección del Motor de Emulación (Mednafen)

La decisión de utilizar `mednafen` se basa en criterios de optimización. Mednafen es una aplicación de línea de comandos (CLI) que carece de interfaces de configuración gráficas. Al recibir los parámetros de ejecución directamente desde la terminal, su uso de memoria es mínimo [4]. Además, su compatibilidad con el entorno de renderizado directo permite decodificar gráficos sin el soporte de administradores de ventanas.

9.3.2. Comunicación Interprocesos en Medio Volátil (`next_game.txt`)

La comunicación asíncrona entre la interfaz visual desarrollada en Python y el orquestador en Bash requiere el almacenamiento de variables. La decisión de ubicar el archivo `next_game.txt` explícitamente en el directorio `/tmp/` responde a la arquitectura del sistema de archivos de Linux. El directorio `/tmp/` se monta como un sistema de archivos temporal (`tmpfs`) alojado directamente en la memoria RAM [1]. Esto previene ciclos de escritura innecesarios en la tarjeta MicroSD, extendiendo su vida útil y garantizando velocidades de lectura y escritura inmediatas.

9.3.3. Orquestación y Control de Flujo (start.sh)

Se seleccionó Bash para programar el ciclo de vida del dispositivo debido a que es el intérprete de comandos integrado en el núcleo, lo que asegura una carga mínima en el procesador. El algoritmo maestro administra el hardware garantizando que el módulo de Python y Mednafen nunca operen simultáneamente, previniendo fallos de caché y la sobrecarga del procesador.

Listing 4: Orquestador principal y bucle maestro de exclusión mutua

```
1 #!/bin/bash
2 export SDL_VIDEODRIVER=kmsdrm
3 export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
4
5 while true; do
6     rm -f /tmp/next_game.txt
7     clear
8
9     # 1. El orquestador pausa su ciclo y ejecuta la interfaz grafica
10    python3 /home/pi/menu.py
11
12    # 2. Verificacion de estado mediante el archivo
13    if [ -f /tmp/next_game.txt ]; then
14        JUEGO=$(cat /tmp/next_game.txt)
15        clear
16
17        # 3. Llamada al emulador absorbiendo la salida estandar de texto
18        mednafen "$JUEGO" > /dev/null 2>&1
19    else
20        break
21    fi
22 done
```

10. Integración de los Componentes en una Solución de Software

La unión de los módulos se consolida en la secuencia de arranque del sistema, donde intervienen los servicios internos del núcleo de Linux para volverlo totalmente autónomo y tolerante a fallos.

10.1. Detección por Eventos de Hardware

Las interrupciones eléctricas originadas al insertar un USB se procesan mediante el gestor udev [2]. La integración se logra con el archivo /etc/udev/rules.d/99-usb-sync.rules:

```
1 ACTION=="add", SUBSYSTEM=="block", KERNEL=="sd[a-z][1-9]", RUN+="/usr/local/
  bin/usb_notify.sh"
```

Esta regla actúa como el puente nativo entre el controlador físico USB y el subprocesso lógico de copiado de archivos.

10.2. Asignación Exclusiva de la Terminal mediante Systemd (`retromenu.service`)

Para automatizar la ejecución al momento de energizar la placa y gestionar la estabilidad del entorno, el módulo orquestador fue encapsulado en la unidad de servicio `retromenu.service`. La estructura de este archivo aísla el control de entrada y salida física.

Listing 5: Servicio de sistema para la gestión del hardware y recuperación de fallos

```
1 [Unit]
2 Description=Menu de Consola Retro en Python
3 After=multi-user.target
4 Conflicts=getty@tty1.service
5
6 [Service]
7 Type=simple
8 User=pi
9 Group=pi
10 StandardInput=tty
11 StandardOutput=tty
12 TTYPath=/dev/tty1
13 ExecStart=/bin/bash /home/pi/start.sh
14 Restart=always
```

El bloque de código implementa configuraciones de alta prioridad:

- **Control de Periféricos (`Conflicts=getty@tty1.service`):** El sistema operativo inicia por defecto un proceso de inicio de sesión (`getty`) en la pantalla primaria. Al declarar un conflicto explícito, el servicio embebido detiene este proceso, tomando el control total de la visualización física en el monitor y reservando las entradas del Gamepad para uso exclusivo de la consola.
- **Enrutamiento Directo (`TTYPath=/dev/tty1`):** Las directivas de salida y entrada estándar fuerzan al servicio a comunicarse con el canal físico 1, asegurando que el video generado por Pygame y Mednafen se muestre en la salida microHDMI sin enviarse a terminales en segundo plano.
- **Tolerancia a Fallos (`Restart=always`):** En entornos embebidos operados por el usuario final, es factible que la ejecución de un juego dañado genere un error en el emulador. Esta instrucción asegura que el sistema recupere automáticamente el ciclo del orquestador, reiniciando la interfaz de forma transparente sin requerir un apagado físico.

11. Conclusiones

La separación del hardware en módulos lógicos independientes permitió aprovechar de mejor manera los recursos limitados de la arquitectura ARM. La configuración directa de las opciones de video sobre el núcleo de Linux y la eliminación de entornos gráficos hicieron posible reducir los tiempos de respuesta y liberar memoria para los emuladores de mayor rendimiento. El intercambio de información mediante archivos evitó la sobrecarga del sistema al asegurar una correcta coordinación entre los procesos de visualización y ejecución. Además, la asignación exclusiva de terminales, junto con mecanismos de recuperación ante fallos y el uso del sistema de detección de eventos de hardware, proporcionó a la tarjeta controladora la autonomía y estabilidad necesarias para operar como un equipo de propósito específico.

12. Cuestionario

1. ¿Qué implicaciones físicas tiene la reducción de voltaje en la fuente de alimentación durante la emulación?

Una fuente inestable induce eventos de subtensión eléctrica en la placa. Para proteger el procesador, el circuito forzará una disminución de la frecuencia operativa. A nivel de software, esto genera un déficit de procesamiento, interrumpiendo el ciclo de reloj de la máquina virtual, manifestándose en latencia gráfica.

2. ¿Por qué se utilizó el archivo `/tmp/next_game.txt` en el orquestador maestro?

Se implementó como un mecanismo de Comunicación Interprocesos (IPC). El menú programado en Python y el emulador son procesos distintos que no comparten espacios de memoria activa. El archivo actúa como un portador temporal que almacena la instrucción del jugador, permitiendo que Python libere por completo la memoria RAM antes de iniciar la emulación. Además, su alojamiento en `/tmp/` protege la memoria de la tarjeta lógica contra escrituras excesivas.

3. ¿Cuál es la función del subsistema `udev` frente a la manipulación de hardware externo?

Actúa como el servicio responsable de procesar las interrupciones de los puertos físicos de entrada. Al detectar tensión y datos, crea el bloque lógico correspondiente y examina las reglas programadas, ejecutando automáticamente los scripts de transferencia sin requerir comandos de teclado.

4. ¿Qué ventaja técnica directa se obtiene al omitir servidores gráficos a favor del driver `KMS/DRM`?

Elimina el tiempo de procesamiento derivado del envío de fotogramas a un servidor intermediario. El sistema de emulación y la interfaz escriben los datos visuales directamente sobre el búfer asignado por el kernel, liberando memoria operativa y garantizando respuestas rápidas durante la interacción del usuario.

5. ¿Cuál es el propósito de declarar un conflicto con el servicio `getty@tty1.service` en la integración?

Asegurar el uso exclusivo de las interfaces de entrada y salida de la tarjeta. Cancelar el servicio de solicitud de credenciales de usuario evita que el sistema operativo intente imprimir texto de consola en la pantalla de forma paralela al software o absorba los comandos direccionales provenientes del mando periférico.

Referencias

- [1] P. Barry y P. Crowley, *Modern Embedded Computing: Designing Connected, Pervasive, Media-Rich Systems*. Waltham, MA: Morgan Kaufmann Publishers, 2012.
- [2] The udev developers, “udev - Linux dynamic device management,” ArchWiki, 2024. [En línea]. Disponible en: <https://wiki.archlinux.org/title/Udev>
- [3] Raspberry Pi Ltd., “Raspberry Pi 4 Model B specifications,” 2024. [En línea]. Disponible en: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>
- [4] Mednafen Team, “Mednafen Emulator Documentation,” GitHub Pages, 2024. [En línea]. Disponible en: <https://mednafen.github.io/documentation/>